03 · Sentiment: how do people feel, and what's changing? (Task 2)

Transcript Intelligence: notebook 3 of 5

The task. Generate a sentiment analysis across call types and identify trends. The brief also asks for more than charts: what the trends mean, what they could indicate, and why someone should care.

Headlines

1. **The Detect outage hit every call type at once**, and customer language on account-manager calls collapsed by about 70%, then recovered after Comply v2 launched.
2. **Support stayed stuck at a low baseline** while the other call types bounced back, and slipped again after the launch, because the fixes for the outage generated a second wave of support tickets. Support sentiment is a lagging indicator of *remediation quality*.
3. **Customers carry the signal.** Staff tone stays steady whatever happens; customer language swings, and tracks the vendor's score more closely.
4. **Internal language was the most negative of any group during the outage**, an early warning about team load that a customer-only view would miss.

How to run. `python -m src.pipeline` (the `sentiment` stage) writes everything this notebook reads. No LLM calls.

1. Setup

In [1]:
```
%load_ext autoreload
%autoreload 2

import sys
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT))

import numpy as np
```

```
import pandas as pd
import plotly.express as px
import plotly.io as pio

from src.load import PROCESSED_DIR, mode_dir
from src.enrich import load_enriched, PERIODS
from src.topics import load_topics, THEME_NAMES
from src.sentiment import load_sentiment, DISAGREE_Z, ARC_THRESHOLD

pio.renderers.default = "plotly_mimetype+notebook_connected"
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_colwidth", 110)
TYPE_COLORS = {"external": "#2a78d6", "support": "#eb6834", "internal": "#8a8f98"}
```

In [2]:

```
sent_scores, call_sent, slices, val = load_sentiment(PROCESSED_DIR)
calls = load_enriched(PROCESSED_DIR)["calls"]
hyb = load_topics(mode_dir("hybrid"))[0]

ORIGINAL_END = pd.Timestamp("2026-04-28").date()
meta = ["call_id", "date", "month", "title", "call_type", "account", "period", "summary", "km_churn_signal"]
S = (call_sent.merge(calls[meta], on="call_id")
              .merge(hyb[["call_id", "primary_theme", "themes"]], on="call_id", how="left"))
S["period"] = pd.Categorical(S["period"].astype(str), PERIODS, ordered=True)
S["is_synthetic"] = S["date"].map(lambda d: pd.Timestamp(d).date() > ORIGINAL_END)
So = S[~S["is_synthetic"]].copy()       # original 100 calls: all headline numbers
print(f"{len(S)} calls scored; analysis uses the {len(So)} original calls unless stated otherwise.")
```

```
115 calls scored; analysis uses the 100 original calls unless stated otherwise.
```

2. Approach: four signals, one cross-check

Signal	Source	Scale	What it's good for
Vendor score	`summary.json` (transcription vendor)	1–5	A holistic judgement of the whole call; the dashboard leads with it
Transcript score	VADER on every sentence, word-weighted	−1 to 1	An independent, transparent check on the vendor score
Customer vs staff	VADER split by who is speaking	−1 to 1	Separates how customers feel from how AegisCloud sounds
Arc	VADER, first third vs last third of each call	change	Did the call get better or worse?

Decisions

- **Don't take the vendor score on trust.** We don't know how it was produced, so it's checked against our own scoring. Calls where the two disagree strongly are flagged for review.
- **VADER as the check, not an LLM.** It's free, instant, deterministic, and anyone can inspect why a sentence scored the way it did. Its weakness (it reads words, not severity) is exactly why it's paired with the vendor score rather than replacing it.
- **Word-weighted averages**, so "Yeah." counts less than a paragraph-long complaint.
- **Business calls only.** A birthday or farewell call scores very high on both measures and would lift internal sentiment for its month. Such calls are screened out before scoring (`01_explore` , section 4), so the trends below describe work conversations.

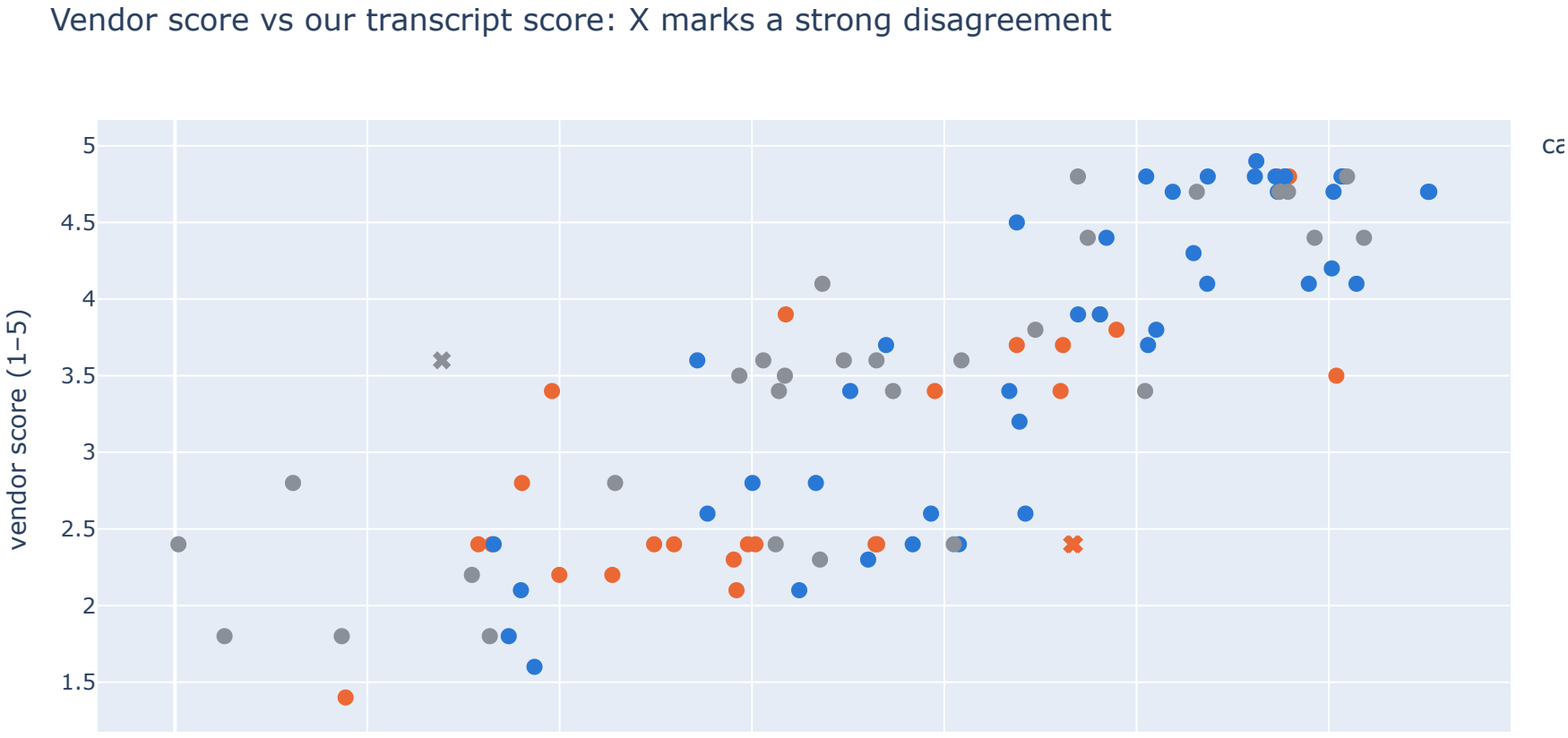
3. Can we trust the vendor's sentiment score?

In [3]:
```
pd.Series({
    "vendor vs transcript score (Pearson)": So["vendor_score"].corr(So["vader_mean"]),
    "vendor vs transcript score (Spearman)": So["vendor_score"].corr(So["vader_mean"], method="spearman"),
    "vendor vs CUSTOMER language": So["vendor_score"].corr(So["customer_vader"]),
    "vendor vs STAFF language": So["vendor_score"].corr(So["staff_vader"]),
}, name="correlation").round(2).to_frame()
```

Out[3]:

	correlation
vendor vs transcript score (Pearson)	0.81
vendor vs transcript score (Spearman)	0.81
vendor vs CUSTOMER language	0.84
vendor vs STAFF language	0.68

In [4]:
```
px.scatter(So, x="vader_mean", y="vendor_score", color="call_type", color_discrete_map=TYPE_COLORS,
           hover_data=["title"], symbol="disagreement", symbol_map={False: "circle", True: "x"},
           labels={"vader_mean": "transcript score (VADER, word-weighted)", "vendor_score": "vendor score (1-5)"},
           title="Vendor score vs our transcript score: X marks a strong disagreement").update_traces(marker_size=9)
```



In [5]:
```
print(f"Flag rule: |z(vendor) − z(transcript)| > {DISAGREE_Z}")
So.loc[So["disagreement"], ["title", "vendor_score", "vader_mean", "customer_vader", "disagreement_note"]] \
  .reset_index(drop=True)
```

```
Flag rule: |z(vendor) − z(transcript)| > 1.5
```

Out[5]:

	title	vendor_score	vader_mean	customer_vader	disagreement_note
0	Support Case #3103 - Silverline Brands Detect False Positives After Patch	2.4	0.466471	0.415656	Transcript language more positive than the vendor score
1	All Hands - April Update	3.6	0.138788	NaN	Vendor score more positive than the transcript language
2	Support Case #7615 - Crestline Wealth Group Policy Sync Delay	2.4	0.467543	0.381695	Transcript language more positive than the vendor score

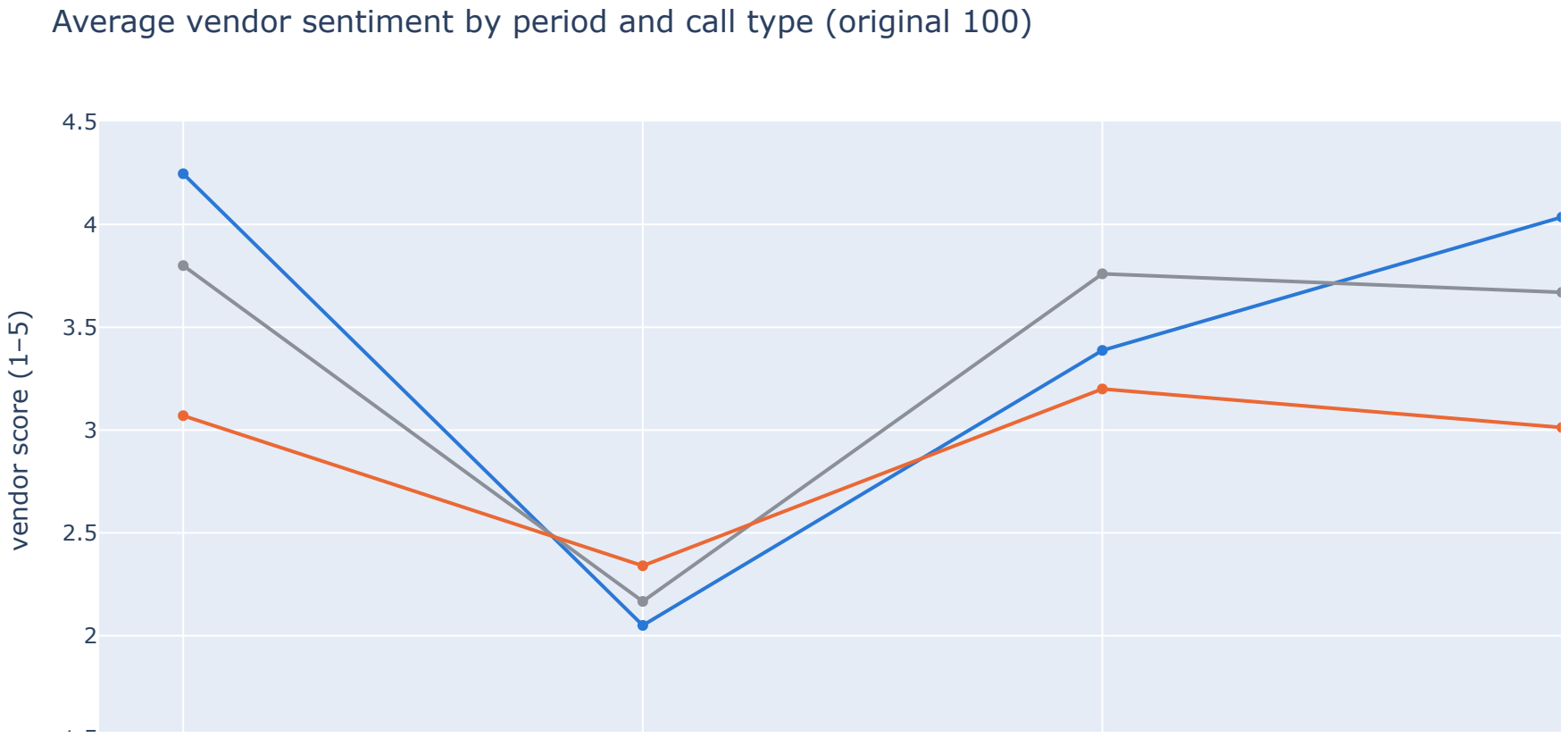
What this shows. The two independent measures agree strongly (r ≈ 0.8), so the vendor score is usable as the headline metric. The few disagreements are instructive rather than errors:

- **Polite customers with serious problems.** For example, a false-positive surge after a patch, or a policy-sync delay with a five-day support gap. The customer is courteous, so the *words* score positive, but the vendor score correctly reads the situation as bad. A word-based score can't see severity.
- **An internal all-hands** reviewing the outage. Its language is full of words like "failure" and "outage", but the meeting itself was constructive, which the vendor score captures.

So each method catches what the other misses. Flagged calls are the ones worth a human read.

4. The headline trend: one incident, every call type

In [6]:
```
trend = (So.groupby(["period", "call_type"], observed=True)["vendor_score"].mean().reset_index())
fig = px.line(trend, x="period", y="vendor_score", color="call_type", markers=True, color_discrete_map=TYPE_COLORS,
              title="Average vendor sentiment by period and call type (original 100)",
              labels={"vendor_score": "vendor score (1–5)"}, category_orders={"period": PERIODS})
fig.update_yaxes(range=[1.5, 4.5])
fig
```



In [7]:
```
tbl = So.pivot_table(index="call_type", columns="period", values="vendor_score", aggfunc="mean", observed=True).round(2)
tbl["outage drop"] = (tbl["Outage"] - tbl["Pre-outage"]).round(2)
tbl["recovered to"] = (tbl["Post-Comply-launch"] / tbl["Pre-outage"]).map("{:.0%} of pre-outage".format)
n = So.pivot_table(index="call_type", columns="period", values="call_id", aggfunc="count", observed=True)
display(tbl)
print("Calls per cell:"); display(n)
```

period	Pre-outage	Outage	Recovery	Post-Comply-launch	outage drop	recovered to
call_type						
external	4.25	2.05	3.39	4.04	-2.20	95% of pre-outage
internal	3.80	2.17	3.76	3.67	-1.63	97% of pre-outage
support	3.07	2.34	3.20	3.01	-0.73	98% of pre-outage

```
Calls per cell:
```

period	Pre-outage	Outage	Recovery	Post-Comply-launch
call_type				
external	15	6	8	14
internal	9	6	5	10
support	10	5	4	8

In [8]:
```
# Customer LANGUAGE on customer-facing calls (staff excluded)
cust = So[So["call_type"] != "internal"].groupby(["period", "call_type"], observed=True)["customer_vader"].mean().unstack()
display(cust.round(3))
ext = cust["external"]
print(f"External calls: customer language fell {1 - ext['Outage'] / ext['Pre-outage']:.0%} during the outage, "
      f"and recovered to {ext['Post-Comply-launch'] / ext['Pre-outage']:.0%} of its pre-outage level after the Comply launch.")
```

call_type	external	support
period		
Pre-outage	0.551	0.334
Outage	0.162	0.233
Recovery	0.454	0.345
Post-Comply-launch	0.533	0.313

```
External calls: customer language fell 71% during the outage, and recovered to 97% of its pre-outage level after the Comply lau
nch.
```

What this shows.

- **The outage moved every call type at the same time.** Customer calls, support tickets *and* internal meetings all dropped to between 2.0 and 2.4 out of 5. No other event in the data does that. For leadership, it means a single reliability failure became a company-wide sentiment event across engineering, customer success and support simultaneously.
- **Account-manager (external) calls fell furthest and recovered fully.** Customer language on these calls fell by about 70%, then returned to pre-outage levels after Comply v2 launched. The recovery lines up with the go-to-market plan to lead with a transparent post-mortem and then a new-product touchpoint. The data is consistent with that plan working.
- **Support is the odd one out.** It starts lowest and falls least (it was never high). It returns only to that low baseline, and while the other call types hold their recovery, support slips again after the Comply launch; customer language on support calls ends below its pre-outage level. The next section explains why.

Caveat: each period × call-type cell holds 4–15 calls, so these are directional, not statistically precise. The pattern is consistent across both independent measures (vendor score and customer language).

5. Why support didn't recover

In [9]:
```
REMEDIATION = r"remediation|rebalanc|circuit breaker|post-outage"
post_support = So[(So["call_type"] == "support") & (So["period"] == "Post-Comply-launch")].copy()
post_support["summary links it to the outage fix"] = post_support["summary"].str.contains(REMEDIATION, case=False)
post_support.sort_values("vendor_score")[["date", "title", "primary_theme", "vendor_score", "customer_vader",
                                          "summary links it to the outage fix"]].reset_index(drop=True)
```

Out[9]:

	date	title	primary_theme	vendor_score	customer_vader	summary links it to the outage fix
0	2026-04-10	Support Case #8179 - Clearwater Medical Overage Charges Dispute	Billing disputes	2.4	0.119324	False
1	2026-04-12	Support Case #9856 - Silverline Brands Backup Window Exceeded	Backup / Protect	2.4	0.082028	True
2	2026-04-15	Support Case #6147 - Summit Trust Detect Alert Delays	Detect outage & fallout	2.4	0.130351	True
3	2026-04-22	Support Case #3536 - Stratos Cloud LogVault Integration Timeout	Detect outage & fallout	2.4	0.270716	True
4	2026-04-27	Support Case #7615 - Crestline Wealth Group Policy Sync Delay	Identity / SSO / MFA	2.4	0.381695	True
5	2026-04-08	Support Case #3296 - Pinnacle Insurance Comply v2 Report Formatting Issue	Comply v2 launch	3.4	0.338362	False
6	2026-04-18	Support Case #7570 - Silverline Brands Comply v2 License Upgrade	Billing disputes	3.9	0.478604	False
7	2026-04-25	Support Case #8811 - Blackridge Investments Custom Compliance Template	Comply v2 launch	4.8	0.705599	False

In [10]:
```
low = post_support[post_support["vendor_score"] < 3]
print(f"{len(low)} of {len(post_support)} post-launch support calls scored below 3; "
      f"{low['summary links it to the outage fix'].sum()} of those are explicitly linked to the outage remediation in their su
for _, r in low[low["summary links it to the outage fix"]].iterrows():
    sentence = next(s for s in r["summary"].split(". ") if pd.Series([s]).str.contains(REMEDIATION, case=False).iat[0])
    print(f"\n• {r['title']}\n  {sentence.strip()}.")
```

```
5 of 8 post-launch support calls scored below 3; 4 of those are explicitly linked to the outage remediation in their summaries.

• Support Case #9856 - Silverline Brands Backup Window Exceeded
  Support engineer David Kim identified a dramatic throughput drop from ~800 MB/s to ~180–220 MB/s, potentially linked to backe
nd network routing changes made during a March remediation effort for a separate module (Aegis Detect).

• Support Case #6147 - Summit Trust Detect Alert Delays
  David Kim identified the root cause as Summit Trust's tenant being placed on an overloaded processing node cluster during pos
t-outage infrastructure remediation in March.

• Support Case #3536 - Stratos Cloud LogVault Integration Timeout
  David Kim and team lead Priya Patel diagnosed the root cause as a circuit breaker payload threshold introduced in the April 8
th remediation build (v4.1.2) being triggered by Stratos's high event volume of 8,000-12,000 events per minute during peak hour
s.

• Support Case #7615 - Crestline Wealth Group Policy Sync Delay
  Support engineer Sarah Chen investigated the tenant logs and identified a sync worker queue backup issue on a shared cluster
tier, potentially related to remediation work done after a mid-March Detect incident.
```

What this shows. Support sentiment didn't stay low by chance. **Most of the unhappy support calls after the launch were caused by the outage fix itself**: tenants rebalanced onto overloaded nodes (alert delays), network routing changes (slower backups), the new circuit breaker (integration timeouts) and shared sync queues (policy delays). Another came from the Comply v2 launch (an overage charge from a billing misconfiguration).

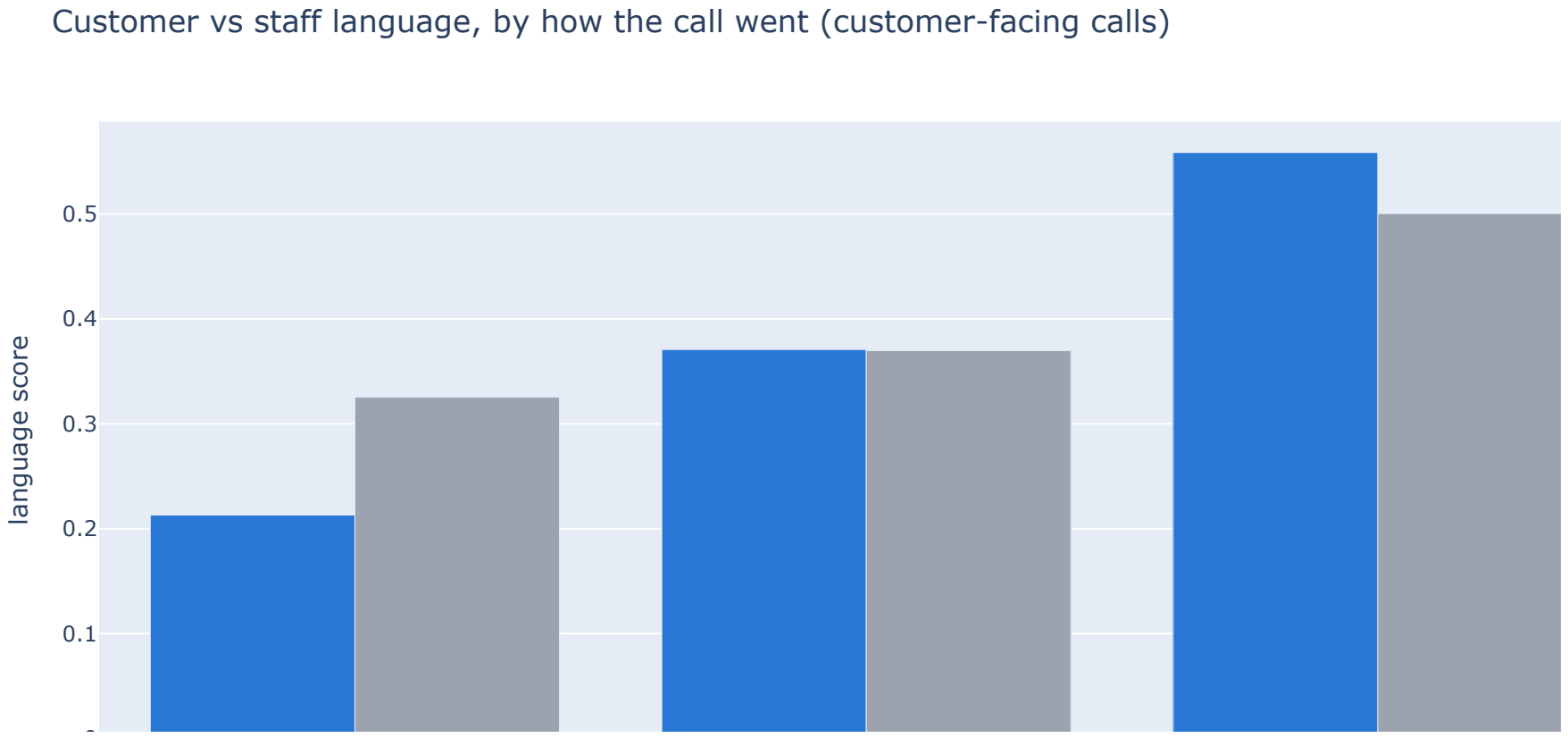
Why it matters.

- **For engineering leaders:** remediation work needs the same change management, regression testing and customer-impact review as feature work. The outage fix created at least four new customer incidents.
- **For support leaders:** support sentiment is a *lagging indicator of remediation quality*. After any major incident, watch for a second wave four to six weeks later and staff for it.
- **For customer success:** several of these customers were already at renewal risk from the outage. A second, self-inflicted problem lands on accounts that have the least trust left.

6. Customers carry the signal

In [11]:
```
side = So[So["call_type"] != "internal"].copy()
side["call mood (vendor score)"] = pd.cut(side["vendor_score"], [0, 2.5, 3.5, 5],
                                          labels=["negative (<2.5)", "mixed", "positive (>3.5)"])
g = side.groupby("call mood (vendor score)", observed=True).agg(
        calls=("call_id", "count"), customer=("customer_vader", "mean"), staff=("staff_vader", "mean"))
g["customer − staff"] = g["customer"] - g["staff"]
display(g.round(3))
px.bar(g.reset_index().melt(id_vars="call mood (vendor score)", value_vars=["customer", "staff"],
                            var_name="speaker", value_name="language score"),
       x="call mood (vendor score)", y="language score", color="speaker", barmode="group",
       color_discrete_map={"customer": "#2a78d6", "staff": "#9CA3AF"},
       title="Customer vs staff language, by how the call went (customer-facing calls)")
```

call mood (vendor score)	calls	customer	staff	customer − staff
negative (<2.5)	23	0.213	0.325	-0.112
mixed	13	0.371	0.370	0.001
positive (>3.5)	34	0.559	0.500	0.058



What this shows. Averaged across all calls, customers and staff sound about the same. That average hides the real pattern:

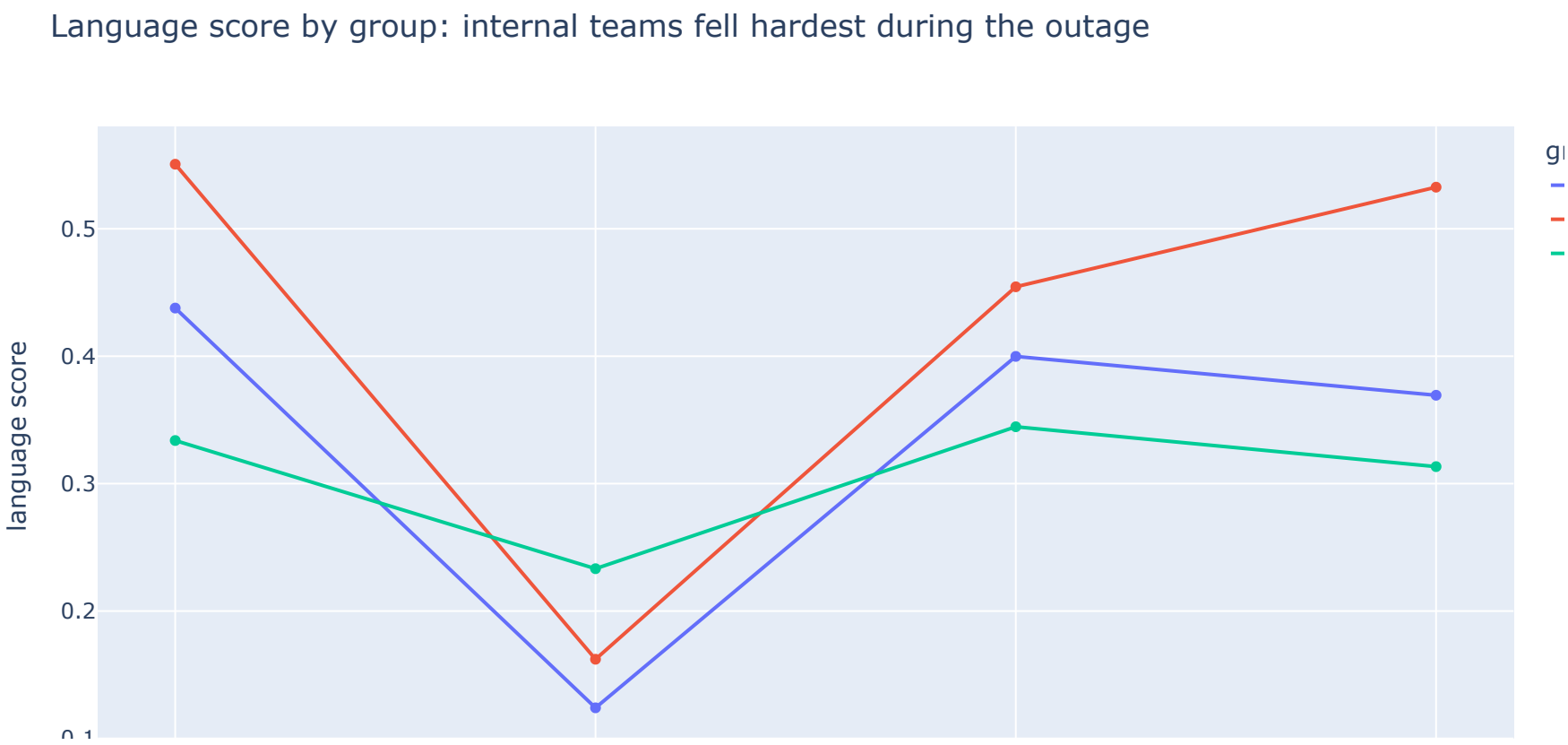
- **Staff tone moves about half as much as customer tone.** It stays professionally positive whether the call is going well or badly.
- **Customer tone swings widely.** It's well below staff on bad calls and above staff on good ones.
- That's why customer language tracks the vendor's overall score more closely than staff language does (section 3).

Why it matters. Staff speak about two-thirds of the words in these calls. A call-level score therefore mostly measures how AegisCloud *sounds*, which hardly varies, and dampens the customer's actual reaction. **Customer-only sentiment is the right metric** for account health and churn risk. `04_insights` and the dashboard's Accounts tab report it per account; the risk score itself uses the vendor score, which tracks customer language closely (r ≈ 0.84, section 3).

7. The internal view: incident load on the team

In [12]:
```
internal = (So[So["call_type"] == "internal"].groupby("period", observed=True)
              .agg(calls=("call_id", "count"), vendor_score=("vendor_score", "mean"), staff_language=("staff_vader", "mean")))
display(internal.round(3))
all_groups = pd.concat([
    So[So["call_type"] == "internal"].groupby("period", observed=True)["staff_vader"].mean().rename("internal staff"),
    side[side["call_type"] == "external"].groupby("period", observed=True)["customer_vader"].mean().rename("customers (externa
    side[side["call_type"] == "support"].groupby("period", observed=True)["customer_vader"].mean().rename("customers (support)
], axis=1)
px.line(all_groups.reset_index().melt(id_vars="period", var_name="group", value_name="language score"),
        x="period", y="language score", color="group", markers=True, category_orders={"period": PERIODS},
        title="Language score by group: internal teams fell hardest during the outage")
```

period	calls	vendor_score	staff_language
Pre-outage	9	3.800	0.438
Outage	6	2.167	0.124
Recovery	5	3.760	0.400
Post-Comply-launch	10	3.670	0.369



What this shows. During the outage, the language in AegisCloud's *own* internal meetings was more negative than anything customers said. Internal sentiment recovered quickly afterwards.

Part of this is vocabulary: war rooms and root-cause reviews are full of words like "failure", "crash" and "outage", which a word-based score reads as negative. But the vendor's holistic score for internal calls falls just as sharply, and the calls themselves show the strain: engineers pulled off Comply v2 for four days, the launch slipping, sprint work deferred.

Why it matters. Customer-facing sentiment tells leadership how the market feels; internal sentiment is an **early warning about team load and burnout**, and it's invisible in a customer-only view. A Transcript Intelligence tool should show both.

8. Within-call arcs: do calls get better?

Each call is split into thirds. The arc compares the average language score in the last third with the first (threshold ±0.10).

In [13]:
```
display(pd.crosstab(So["call_type"], So["arc"], margins=True))
display(So.groupby("call_type")[["arc_delta", "customer_arc_delta"]].mean().round(3)
          .rename(columns={"arc_delta": "whole call: last − first third", "customer_arc_delta": "customer only"}))
side_arc = So.assign(start=pd.cut(So["vendor_score"], [0, 2.5, 3.5, 5], labels=["negative", "mixed", "positive"])) \
             .groupby("start", observed=True)[["arc_delta", "customer_arc_delta"]].mean().round(3)
side_arc.columns = ["whole call: last − first third", "customer only: last − first third"]
side_arc
```

arc	declined	flat	improved	All
call_type				
external	4	20	19	43
internal	4	10	16	30
support	1	13	13	27
All	9	43	48	100

	whole call: last − first third	customer only
call_type		
external	0.116	0.122
internal	0.123	NaN
support	0.098	0.172

Out[13]:

start	whole call: last − first third	customer only: last − first third
negative	0.183	0.195
mixed	0.140	0.202
positive	0.058	0.081

In [14]:
```
print(f"Calls the vendor tagged with a 'positive_pivot' moment: {val['n_with_pivot']}")
print(f"  share whose language actually improved: {val['share_improved_with_pivot']:.0%}")
print(f"  share improving WITHOUT a pivot tag:    {val['share_improved_without_pivot']:.0%}")
So.sort_values("arc_delta", ascending=False)[["title", "call_type", "vendor_score", "arc_first", "arc_last", "arc_delta"]] \
  .head(5).round(2).reset_index(drop=True)
```

```
Calls the vendor tagged with a 'positive_pivot' moment: 82
  share whose language actually improved: 49%
  share improving WITHOUT a pivot tag:    48%
```

Out[14]:

	title	call_type	vendor_score	arc_first	arc_last	arc_delta
0	Aegis / Nova Retail Group - Renewal Discussion	external	2.6	-0.06	0.54	0.59
1	Aegis / Helix Data - Post-Incident Review	external	2.3	0.10	0.59	0.49
2	INCIDENT: Detect Pipeline Failure - War Room	internal	1.8	-0.17	0.31	0.48
3	Competitive Threat Assessment - Post Outage	internal	2.3	0.05	0.51	0.47
4	Support Case #5889 - Ridgeline Logistics Detect Latency Issues	support	2.4	0.01	0.47	0.47

What this shows.

- **About half of calls end better than they start; few get worse.** The worst calls improve the most. The calls with the biggest turnarounds are hard conversations: a renewal under competitive threat, a post-incident review, the war room itself.
- **Customers in support calls show the biggest in-call lift of any call type.** Agents are resolving or defusing issues within the call, even though the overall support score stays low because the starting point is so low.

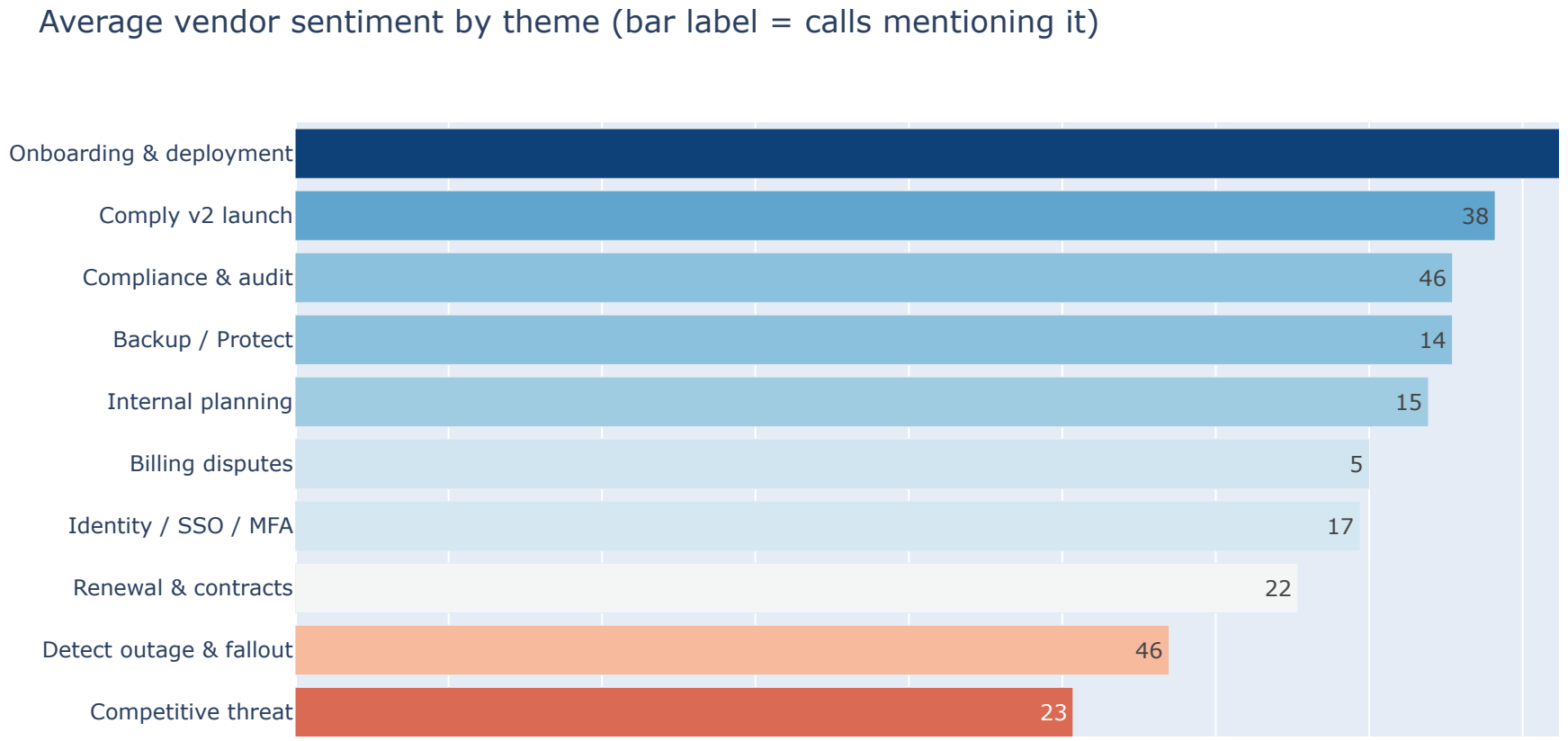
- **The vendor's `positive_pivot` tags don't predict an actual improvement in language**: 49% of tagged calls improved, against 50% of untagged ones. They're useful as pointers to specific moments, but shouldn't be used as a measure of call outcome.

Caveat: some of the "worst calls improve most" effect is regression to the mean, since a call that starts at its lowest point has room to rise. It's still a useful coaching view: which reps reliably turn hard calls around?

9. Which subjects hurt most?

In [15]:
```
th = So.explode("themes").groupby("themes").agg(
        calls=("call_id", "nunique"), vendor=("vendor_score", "mean"),
        customer=("customer_vader", "mean"), improved=("arc", lambda a: (a == "improved").mean()))
th = th.reindex(THEME_NAMES).sort_values("vendor")
display(th.round(2))
px.bar(th.reset_index(), x="vendor", y="themes", orientation="h", text="calls",
       title="Average vendor sentiment by theme (bar label = calls mentioning it)",
       labels={"vendor": "vendor score (1–5)", "themes": ""}, color="vendor",
       color_continuous_scale="RdBu", range_color=[2, 4.5]).update_layout(coloraxis_showscale=False)
```

themes	calls	vendor	customer	improved
Competitive threat	23	2.53	0.30	0.70
Detect outage & fallout	46	2.85	0.32	0.65
Renewal & contracts	22	3.27	0.43	0.50
Identity / SSO / MFA	17	3.47	0.43	0.24
Billing disputes	5	3.50	0.36	0.40
Internal planning	15	3.69	NaN	0.20
Backup / Protect	14	3.77	0.43	0.50
Compliance & audit	46	3.77	0.47	0.46
Comply v2 launch	38	3.91	0.53	0.47
Onboarding & deployment	13	4.42	0.56	0.31



In [16]:
```
churn = So.groupby(So["km_churn_signal"] > 0).agg(calls=("call_id", "count"), vendor=("vendor_score", "mean"),
                                                  customer=("customer_vader", "mean"))
churn.index = churn.index.map({False: "no churn signal", True: "churn signal tagged"})
churn.round(2)
```

Out[16]:

km_churn_signal	calls	vendor	customer
no churn signal	48	4.16	0.56
churn signal tagged	52	2.72	0.30

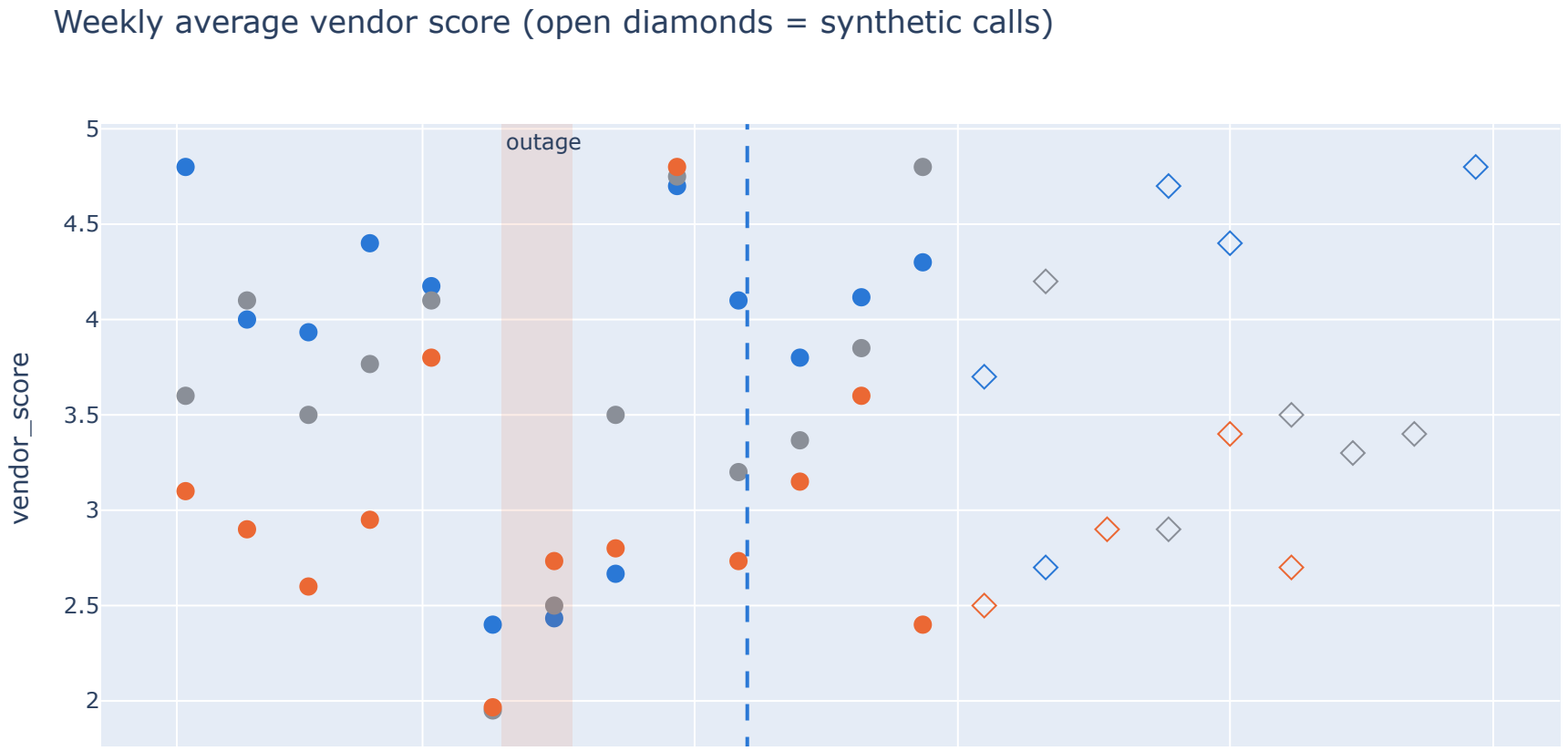
What this shows. Competitive threat and **the outage** are the most negative subjects, and **onboarding** is the most positive: new customers are at their happiest at kickoff. Calls where competitors come up are also among the most likely to *improve* during the call. When an account manager engages directly with a competitive threat, the conversation often ends better than it started.

Calls with a vendor-tagged churn signal score far lower on both measures, which supports using sentiment as an input to churn risk (`04_insights`).

10. The extended timeline (with the later, synthetic calls)

Weekly averages with the synthetic transcripts included, shown separately because they're generated, not observed.

In [17]:
```
wk = (S.assign(week=pd.to_datetime(S["date"]).dt.to_period("W").dt.start_time)
        .groupby(["week", "call_type", "is_synthetic"])["vendor_score"].mean().reset_index())
fig = px.scatter(wk, x="week", y="vendor_score", color="call_type", symbol="is_synthetic",
                 color_discrete_map=TYPE_COLORS, symbol_map={False: "circle", True: "diamond-open"},
                 title="Weekly average vendor score (open diamonds = synthetic calls)")
fig.add_vrect(x0="2026-03-10", x1="2026-03-18", fillcolor="#eb6834", opacity=0.12, line_width=0,
              annotation_text="outage", annotation_position="top left")
fig.add_vline(x="2026-04-07", line_dash="dash", line_color="#2a78d6")
fig.update_traces(marker_size=10)
fig
```



11. So what? Recommendations by stakeholder

Stakeholder	Finding	Recommendation
Engineering leadership	The outage fix caused a second wave of customer incidents	Treat remediation like a release: regression tests, staged rollout, customer-impact review

Stakeholder	Finding	Recommendation
Support leadership	Support stayed at its low baseline and slipped again after launch, driven by fix side effects	Use support sentiment as a lagging indicator of fix quality; plan capacity for a post-incident second wave
Customer success & sales	Customer language fell ~70% in the outage; recovered after the Comply launch	Pair incident transparency with a forward-looking product touchpoint; track **customer-only** sentiment per account
Sales managers	Competitive-threat calls are the most negative but often turn around in-call	Build a competitive-objection playbook from the calls that improved most
Executives	One incident moved every call type at once, including internal morale	Report internal and customer sentiment side by side; internal sentiment is an early warning for team load
Product	Onboarding is the happiest moment; Identity calls rarely improve in-call	Protect onboarding quality; investigate why Identity issues can't be resolved on the call

12. Limitations

- **VADER reads words, not meaning:** it misses sarcasm, severity and domain language ("outage" is always negative). That's why it's a check on the vendor score, not a replacement.
- **The vendor score's method is unknown.** It agrees well with our scoring, but it's still a black box.
- **Transcripts cover about a third of each meeting** (`01_explore`), so call-level scores are samples.
- **Small cells.** Period × call-type groups hold 4–15 calls. The trends are directional and are supported by two independent measures, but they aren't statistically precise.
- **Synthetic calls** are excluded from every headline number and shown separately in section 10.

Next: `04_insights.ipynb`